

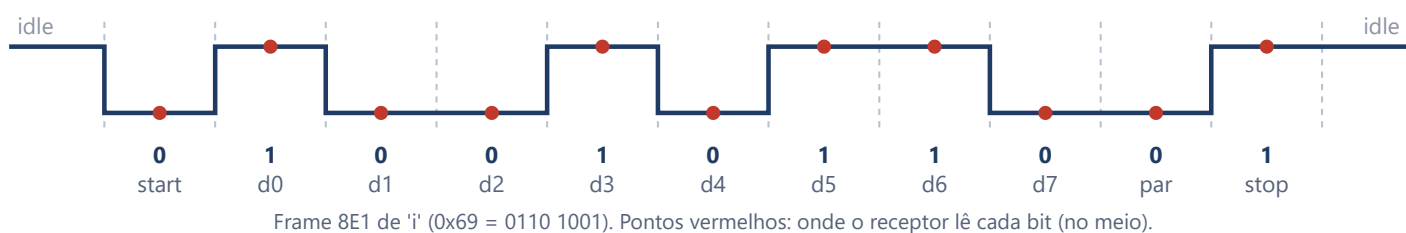
Comunicação serial UART: consulta

- **Tempo de bit:** $T = 1/\text{baud}$ (9600 $\rightarrow$ 104,2 μs)
- **Bits no frame:** $n = \text{start} + \text{dados} + \text{paridade} + \text{stop}$
- **Tempo de frame:** $n \cdot T$
- **Bytes por segundo:** baud / n
- **Tempo para N bytes:** $N \cdot n / \text{baud}$
- **Eficiência:** bits de dado / n
- **Amostra do bit k** (start = 0): $(k + \frac{1}{2}) \cdot T$
- **Tolerância de relógio:** $|\epsilon| < 0,5 / (n - 0,5)$
- **Baud pelo osciloscópio:** 1 / (menor pulso)
- **Paridade par:** total de 1s (dado + paridade) é par

1. Conceitos básicos

- **Serial:** um bit de cada vez num único fio. Usa poucos fios e alcança distâncias maiores. A **paralela** manda vários bits ao mesmo tempo em vários fios, mas sofre com bits chegando em instantes diferentes (skew) e interferência entre fios.
- **Síncrona:** há um **clock compartilhado**, e o receptor lê o dado no ritmo desse clock (ex.: SPI, I²C).
- **Assíncrona: sem clock compartilhado.** Os dois lados combinam o baud antes, cada um usa o próprio relógio, e o receptor se sincroniza **a cada frame** pelo start bit. Os frames podem chegar a qualquer momento (ex.: UART).
- **Simplex:** um só sentido. **Half-duplex:** os dois sentidos, um de cada vez. **Full-duplex:** os dois ao mesmo tempo. A UART com TX e RX é full-duplex.
- **UART** (Universal Asynchronous Receiver/Transmitter) transforma byte em sequência de bits no tempo (serializa) e faz o caminho inverso (desserializa).
- **Ligação:** TX de um no RX do outro (cruzado) e **GND comum**. Sem o terra comum, os dois lados não têm a mesma referência de tensão.
- **Bit-banging:** gerar ou ler o sinal por software, num pino comum, controlando o tempo no programa.

2. O frame



Parte	Valor	Função
Idle	1	Linha parada em nível alto quando não há transmissão.
Start	0	A borda de descida 1 $\rightarrow$ 0 avisa que começou um frame e é a referência de tempo do receptor.
Dados	5 a 9 bits	O conteúdo (normalmente 8 bits), enviado LSB primeiro .
Paridade	0/1	Opcional; serve para detectar erro. Pode ser par (E), ímpar (O) ou nenhuma (N).
Stop	1	Devolve a linha ao repouso, garantindo uma borda no próximo start. Se for lido como 0, é erro de frame .

- **Notação:** 8N1 = 8 dados, sem paridade, 1 stop (10 bits). 8E1 = 8 dados, paridade par, 1 stop (11 bits).
- **Por que o idle é 1?** Assim o start sempre gera uma borda clara. E um fio rompido ou em curto com o GND (linha em 0) aparece como erro, em vez de passar por repouso.
- **Por que existe o stop?** Sem ele, se o último bit fosse 0, o start seguinte (também 0) não geraria borda.
- **LSB primeiro** é só convenção; os dois lados precisam combinar. Ler na ordem errada espelha o byte (0x69 vira 0x96).

3. Baud, tempo e eficiência

- **Baud** = símbolos por segundo. Na UART, cada símbolo é 1 bit, então baud = bits/s. Mas o baud conta **todos** os bits do frame: 9600 baud em 8N1 dão 960 bytes/s, e não 1200.

Baud	T	Frame 8N1	Frame 8E1	B/s 8N1
4800	208,3 μs	2,08 ms	2,29 ms	480
9600	104,2 μs	1,04 ms	1,15 ms	960
19200	52,1 μs	0,52 ms	0,57 ms	1920
115200	8,68 μs	86,8 μs	95,5 μs	11520

Formato	n	Eficiência
7E1	10	70%
8N1	10	80%
8E1 / 8N2	11	72,7%
8E2	12	66,7%

- 1 frame 8E1 a 9600: $11 \times 104,2 \mu s = 1,146 \text{ ms}$
- "Hello" (5 bytes), 8N1, 9600: $5 \times 10 / 9600 = 5,2 \text{ ms}$
- 1024 bytes, 8N1, 115200: $1024 \times 10 / 115200 = 88,9 \text{ ms}$
- Menor pulso no osciloscópio = 26 μs : $1/26 \mu s \approx 38\,460 \rightarrow 38400 \text{ baud}$ (pulsos largos = vários bits iguais seguidos)
- Frame 8E1 medido com 2,29 ms: $11 / 2,29 \text{ ms} \approx 4800 \text{ baud}$
- Taxa útil 8E1 a 9600: $9600 \times 8/11 \approx 6982 \text{ bit/s}$ de dado $\approx 873 \text{ B/s}$
- Leitura da paridade (k = 9), 8E1, 9600: $9,5 \times 104,2 \mu s \approx 990 \mu s$ após a borda do start

4. Como o receptor lê os bits

1. Espera a linha cair de 1 para 0. Esse é o início do start e vira o "zero" do relógio do frame.
 2. Em **T/2** relê a linha. Se ainda está em 0, o start é válido; se já voltou a 1, era ruído e ele descarta.
 3. Lê um bit a cada T, sempre no **meio**: em 1,5T, 2,5T, ...
 4. Confere o stop (precisa ser 1) e depois a paridade.
- **Por que no meio?** Na borda o sinal está mudando e a leitura é incerta. No meio ele está estável, e sobra folga de $\pm T/2$ para pequenos erros de tempo.
 - **Diferença de relógio:** os relógios do TX e do RX nunca são idênticos, e o erro **se acumula** ao longo do frame. A comunicação funciona enquanto o desvio no último bit for menor que T/2: até $\approx 5,3\%$ em 8N1 e $\approx 4,8\%$ em 8E1 (na prática, $< 2\%$). O erro **zera a cada start**, e é isso que permite funcionar sem clock compartilhado.
 - **A tolerância percentual não depende do baud**, só do número de bits do frame (frames maiores toleram menos). Com baud alto, o que diminui é a folga em μs .
 - **Em software:** calcular o instante do bit i a partir do início do frame ($t_0 + i \cdot T$), e não como "bit anterior + T", para os atrasos não se somarem. Prefira medir o tempo sem travar o processador (em vez de usar `delay`).

5. Erros

- **Paridade par:** o bit de paridade é escolhido para o total de 1s ficar par. Se o dado tem um número ímpar de 1s, a paridade é 1; se tem um número par, é 0. A **ímpar** é sempre o inverso. O receptor refaz a conta e compara com o bit que chegou.

Byte	Binário	1s	Par	Ímpar	Situação	Paridade detecta?
'A' 0x41	0100 0001	2	0	1	1 bit errado (ou 3, 5...)	Sim
'C' 0x43	0100 0011	3	1	0	2 bits errados (ou 4...)	Não: a contagem volta a ser par
'U' 0x55	0101 0101	4	0	1	O próprio bit de paridade virou	Sim, e o dado (que estava certo) é descartado
0x00	0000 0000	0	0	1		
0xFF	1111 1111	8	0	1	Qual bit errou / corrigir	Não; só sabe que há erro

Erro	Como se percebe	Causas típicas
Paridade	Paridade recalculada $\neq$ recebida	Ruído inverteu um bit; configuração de paridade diferente nos dois lados
Frame	Stop lido como 0	Baud ou formato diferente, relógio muito fora, ruído, linha presa em 0
Break	Linha em 0 por mais de um frame inteiro	Fio rompido ou em curto com GND (às vezes é usado de propósito como sinal)
Falso start	Start já voltou a 1 na releitura em T/2	Pulso curto de ruído na linha em repouso; deve ser ignorado

Ordem de conferência: primeiro o stop, depois a paridade. Se o stop está errado, os bits foram lidos fora de lugar e a paridade não significa nada. Para detectar mais erros (ou corrigi-los), usam-se CRC ou Hamming, em camadas acima.

6. Lendo um frame no fio

```
No fio (8E1): 0 | 1 0 0 1 0 1 1 0 | 0 | 1      (start | d0..d7 | paridade | stop)
dados d0..d7 = 1 0 0 1 0 1 1 0 → inverter a ordem: 0110 1001 = 0x69 = 'i'
número de 1s = 4 (par) → paridade par esperada 0, recebida 0 → OK; stop = 1 → OK
Com d3 virado:      1 0 0 0 0 1 1 0 → 0x61 'a'; três 1s → esperada 1, recebida 0 → ERRO DE PARIDADE
Com d3 e d5 virados: 1 0 0 0 0 0 1 0 → 0x41 'A'; dois 1s → esperada 0, recebida 0 → passa como OK (não detectado)
Stop lido como 0      → ERRO DE FRAME
```

Dica: **'U' (0x55)** no fio vira uma onda quadrada (0 1 0 1 ...), boa para medir o baud. No analisador lógico, o decodificador precisa estar configurado igual ao TX (baud, bits, paridade, stop, idle alto). Se a onda está certa mas o texto sai errado, a configuração está diferente.

7. Casos conceituais

P. Como o RX sabe quando ler cada bit, sem clock? Os dois lados combinam o baud antes. O RX usa a descida do start como instante zero e lê o bit k em $(k + \frac{1}{2}) \cdot T$ com o próprio relógio, ressinchronizando a cada frame.

P. TX a 9600 e RX a 19200 (ou 4800)? O RX lê nos instantes errados. Resultado: bytes errados (lixo no monitor), erros de frame e de paridade. É o sintoma clássico de baud diferente.

P. Relógios 1% diferentes é problema? E 8%? 1%, não: no último bit o desvio é $\approx 0,1T$. 8%, sim: $9,5 \times 8\% \approx 0,76T > 0,5T$, e os últimos bits são lidos no bit vizinho.

P. Aumentar o baud aumenta a tolerância? Não. A tolerância percentual depende só do tamanho do frame. Com baud alto, a folga em μs fica menor e atrasos fixos pesam mais.

P. Por que ler no meio do bit? Na borda o sinal está em transição. No meio está estável, com folga de $\pm T/2$.

P. Por que não usar `delay()` ? Trava o processador, tem resolução grosseira e, se cada espera é medida a partir da anterior, os atrasos se somam ao longo do frame.

P. Dado 0x00: como distinguir de "nada"? O repouso é 1. 0x00 vem dentro de um frame (start, 8 zeros, paridade e stop = 1). Linha em 0 por mais de um frame inteiro é break.

P. Para que 2 stop bits? Dar mais tempo de repouso a receptores lentos. Custa eficiência e não melhora a tolerância de relógio.

P. Dois bits viraram. A paridade pega? Não. A paridade só detecta número ímpar de erros.

P. Erro de paridade $\times$ erro de frame? Paridade: o frame chegou bem formado, mas o conteúdo não confere (bit virou). Frame: o stop veio 0, ou seja, o receptor perdeu o alinhamento.

P. TX em 8E1 e RX esperando 8N1? O RX trata a paridade como stop. Quando ela é 0, dá erro de frame; quando é 1, aceita. Alguns bytes passam e outros dão erro.

P. Pulso curto de ruído na linha parada? O RX relê o start em $T/2$, vê 1 e descarta. Nenhum byte falso.

P. Linha presa em 0? Start + zeros + stop 0 = erro de frame (break). O RX deve esperar a linha voltar a 1 antes de aceitar outro start.

P. Por que GND comum? Tensão só tem significado em relação a uma referência; sem terra comum, os níveis lidos são indefinidos.

P. TX ligado em TX? Ninguém escuta, e as duas saídas ficam em curto entre si.

P. Pino de entrada solto? Fica flutuando e lê valores aleatórios, gerando falsos starts. Um pull-up mantém a linha em 1 (repouso).

P. UART $\times$ SPI/I²C? A UART é assíncrona (sem clock, baud combinado, ponto a ponto). SPI e I²C são síncronas (o mestre fornece o clock), então não sofrem com diferença de relógio, mas usam fio de clock.